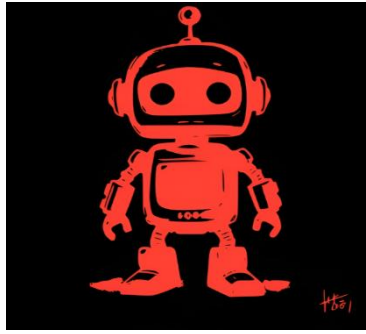




AEGIS

The Alchemists

Alchemist	Student Number
Kutlwano Tau	u24579484
Myron Wheeler	u24579565
Sambulo Zikhali	u24616592
Luyanda Ndlovu	u24580717
Lusanda Mthembu	u23602016

SRS DOCUMENT

COS301 AEGIS TEAM
UNIVERSITY OF PRETORIA
PRETORIA 012

Contents

AEGIS Deployment Topology/Arrangement.....	2
Component Selection & Comparative Analysis.....	2
Backend Orchestration Layer: FastAPI and Piston on AWS EC2.....	2
Operational Deployment Specifications.....	2
Ci/CD pipeline & deployment.....	2
Environment Parity Configuration.....	3
Secrets Management Protocol.....	3
Deployment Diagrams.....	4

AEGIS DEPLOYMENT TOPOLOGY/ARRANGEMENT

The AEGIS system uses a Distributed Cloud Architecture by exploiting production-ready free-tier platforms. This architecture isolates the presentation layer, application orchestration backend, data persistence, and untrusted code execution sandbox to ensure decoupling and modularity.

COMPONENT SELECTION & COMPARATIVE ANALYSIS

Frontend Hosting: Vercel

- **Approach:** Serverless deployment by using Vercel's Edge Network.

BACKEND ORCHESTRATION LAYER: FASTAPI AND PISTON ON AWS EC2

- **Approach:** Containerised deployment of the FastAPI application backend. Data Persistence Layer: Supabase (Managed PostgreSQL)

OPERATIONAL DEPLOYMENT SPECIFICATIONS

CI/CD PIPELINE & DEPLOYMENT

System deployment is fully automated via GitHub Actions (backend-deployment.yml) upon code pushes or pull request merges to tracked branches (main, dev).

1. **Container Build & Registry Push:** GitHub Actions builds a lightweight Docker image, sanitizes the active Git branch into a valid image tag, and pushes the artifact to Amazon Elastic Container Registry.
2. **SSH Remote Deployment:** The workflow opens an SSH connection to the EC2 Elastic IP host using stored secrets.
3. **Container Lifecycle Management:** The host logs into Amazon ECR, pulls the updated container image, stops existing instances, and spins up new container runtimes on the private aegis-net Docker network.

ENVIRONMENT PARITY CONFIGURATION

Operational contexts are strictly isolated through automated branch routing:

- **Production (main branch):** Deploys container runtime named myapp-main mapped internally to host port 8080 (8080:8000).
- **Development (dev branch):** Deploys a container runtime named myapp-dev mapped internally to host port 8081 (8081:8000).
- **Nginx Autoconfiguration:** The deployment workflow dynamically generates and injects `/etc/nginx/conf.d/fastapi.conf`, binding incoming public domain requests directly to the designated internal port before executing a zero-downtime Nginx reload.

SECRETS MANAGEMENT PROTOCOL

- Plaintext credentials, connection strings, and private SSH keys are excluded from version control.
- Production credentials are injected at runtime via GitHub Repository Secrets and environment flags.

DEPLOYMENT DIAGRAMS

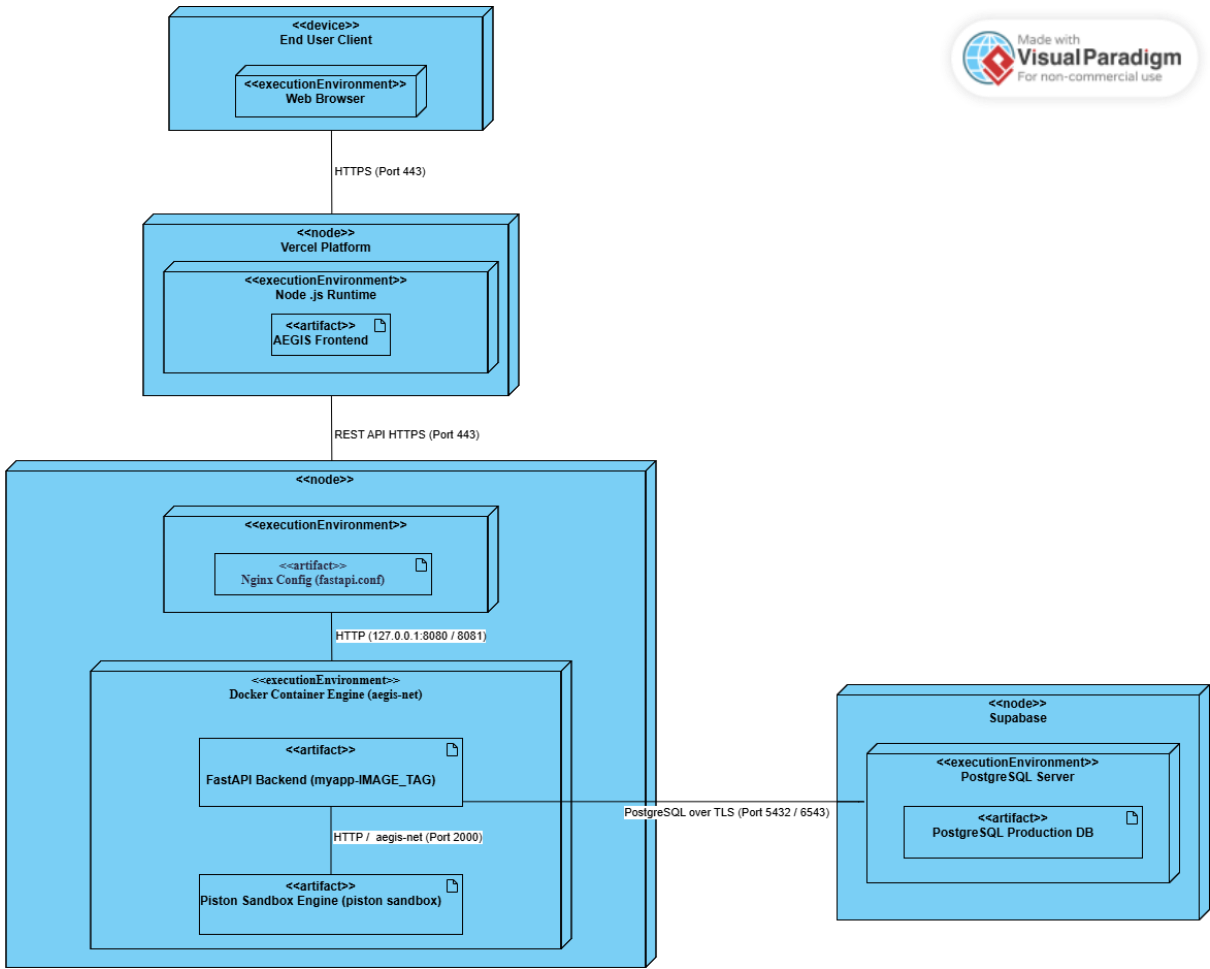


Figure 1: Deployment Diagrams